# Connection Pool, 어떻게 동작하고 어떻게 설정해야 할까?

**DBCP와 HikariCP 차이부터 적정 Pool 사이즈 공식까지**




8기 백엔드 로지

# 목차

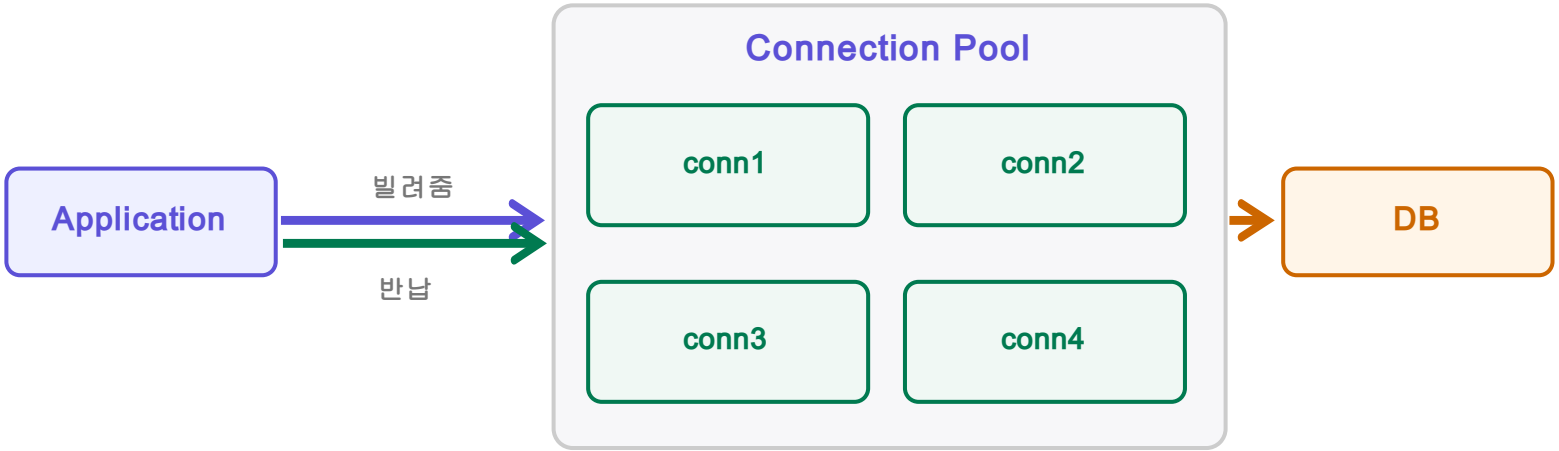
1. Connection Pool 개요
2. DBCP1의 등장 배경 & 문제점
3. DBCP2의 등장 배경 & 문제점
4. HikariCP의 등장 배경
5. HikariCP의 내부 동작 방식
6. HikariCP의 적정 Pool 사이즈 공식

# 1. Connection Pool 개요

# Connection Pool이란?



데이터베이스와 애플리케이션 간의 연결을 미리 여러 개 생성하여 풀(Pool)에 보관해두고,
요청이 있을 때마다 빌려주고 반납받아 재사용하는 기법



# Connection Pool이 필요한 이유



매 요청마다 연결을 새로 만들면 비싸다

## DB 연결 생성 과정

1. **TCP 3-way handshake**
   SYN → SYN / ACK → ACK
2. **DB 인증**
   사용자명 / 비밀번호 검증
3. **세션 초기화**
   메모리 공간, 권한, 트랜잭션 상태 준비

## 연결 비용 예시

사용자 요청

↓

DB 연결 생성 (20ms) ← 대부분의 시간 소비

↓

INSERT 쿼리 (1ms) ← 실제 원하는 작업

↓

DB 연결 해제 (5ms)

# Connection Pool 동작 방식

TCP 3-way handshake는 최초 1번만!



① 사용자 요청 → ② Pool에서 연결 꺼냄 → ③ 쿼리 실행 → ④ Pool에 반납

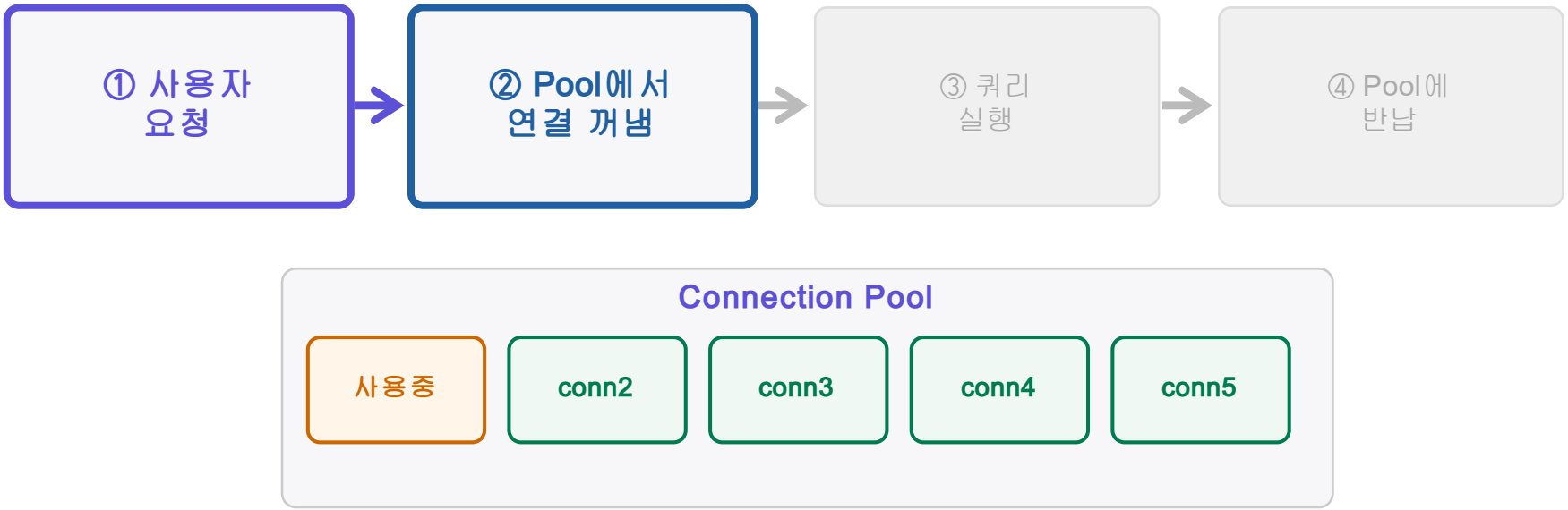
Connection Pool

conn1 | conn2 | conn3 | conn4 | conn5

# Connection Pool 동작 방식

TCP 3-way handshake는 최초 1번만!

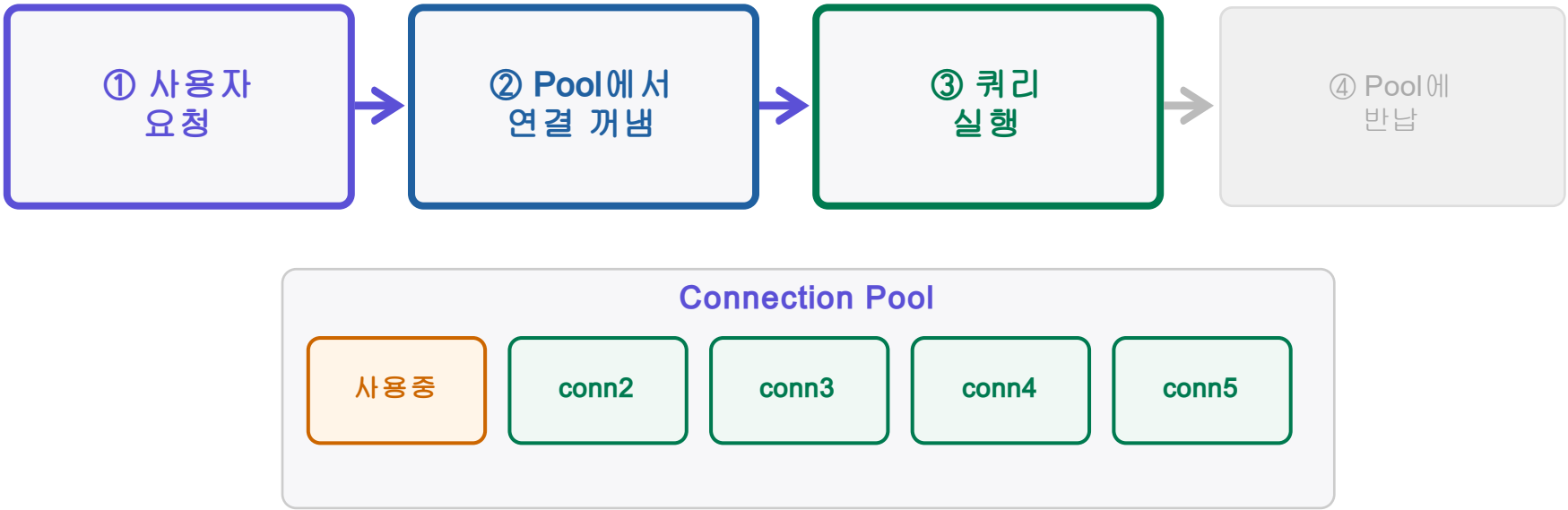




# Connection Pool 동작 방식

TCP 3-way handshake는 최초 1번만!

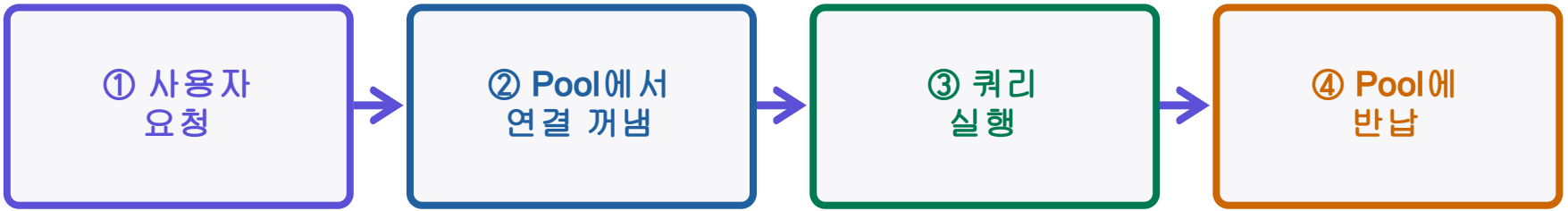


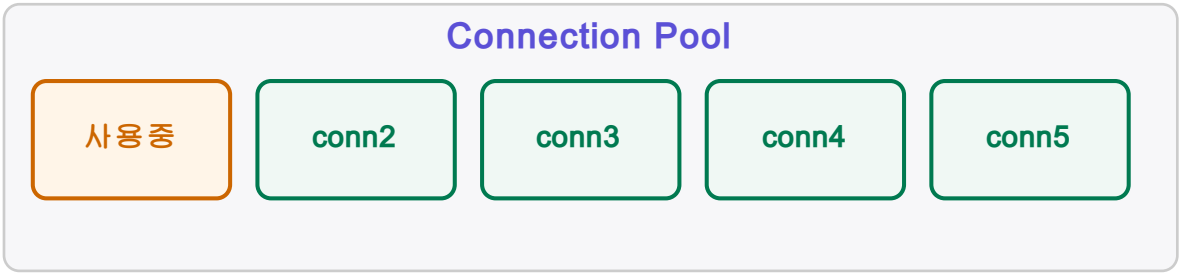


# Connection Pool 동작 방식

TCP 3-way handshake는 최초 1번만!





# 2. DBCP1의 등장 배경 & 문제점

# DBCP1 등장

Apache Commons - Java 최초의 Connection Pool 라이브러리



출시

## 2002년

Apache Commons 프로젝트

기반

## Java 6

JDBC 4.0 호환 마지막 버전(1.4)

의의

## DBCP 표준

톰캣 기본 탑재, Connection Pool 표준

# DBCP1 문제점 ① 단일 Pool Lock



연결이 10개인데 한 번에 1명씩만 꺼낼 수 있다

| | | | | |
|---|---|---|---|---|
| conn1 | conn2 | conn3 | conn4 | conn5 |
| conn6 | conn7 | conn8 | conn9 | conn10 |

**Thread A**

연결 꺼내는 중

**Thread B**

대기

**Thread C**

대기

# DBCP1 문제점 ① 단일 Pool Lock



Thread A가 꺼내는 순간 전체 Pool에 Lock - 나머지 Thread 전부 대기

| | | | | |
|---|---|---|---|---|
| LOCK conn1 | LOCK conn2 | LOCK conn3 | LOCK conn4 | LOCK conn5 |
| LOCK conn6 | LOCK conn7 | LOCK conn8 | LOCK conn9 | LOCK conn10 |

**Thread A**

작업 중

**Thread B**

대기

**Thread C**

대기

# DBCP1 문제점 ① 단일 Pool Lock



Thread A 반납 후에야 Thread B 시작 - 불필요한 직렬화 발생!

| | | | | |
|---|---|---|---|---|
| LOCK conn1 | LOCK conn2 | LOCK conn3 | LOCK conn4 | LOCK conn5 |
| LOCK conn6 | LOCK conn7 | LOCK conn8 | LOCK conn9 | LOCK conn10 |

**Thread A**

반납 완료

**Thread B**

작업 시작!

**Thread C**

대기

# DBCP1 문제점 ②③④



### ② 비효율적 연결 검증

연결을 빌릴 때마다 SELECT 1 쿼리 실행

매번 실제 DB에 쿼리를 날려야 함

불필요한 오버헤드 발생

```
SELECT 1;  // 매 요청마다 실행됨
```

### ③ 메모리 누수 / ④ 레거시

반납 안 된 연결 (Abandoned Connection) 누수

JDBC 드라이버 종료 시 제대로 해제 안 됨

시간이 지날수록 메모리 점점 소모

**OutOfMemoryError 발생 가능**

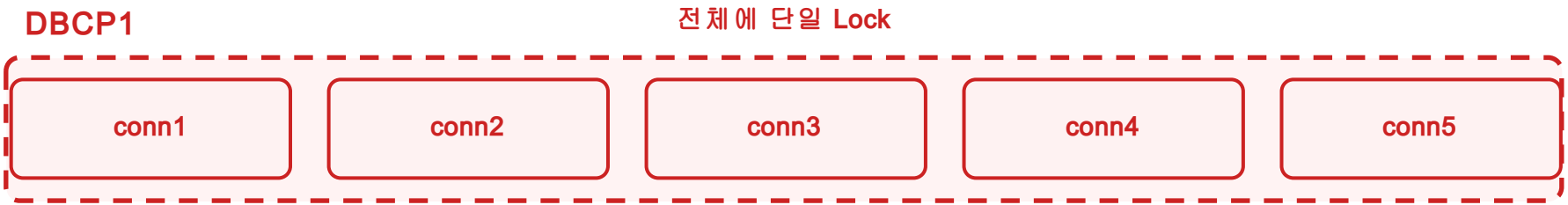
Java 6 기반 레거시 — 유지보수 어려움

# 3. DBCP2의 등장 배경 & 문제점

# DBCP2 - 세분화된 Lock

## DBCP1 - 전체 Pool에 하나의 Lock






Thread A가 연결을 꺼낼 때 → 나머지 9개도 같이 잠김

Thread B~Z 전부 대기 → OS가 스레드 잠재움 (Context Switching 발생)

CPU 코어가 많을수록 더 심각해짐

# DBCP2 - 세분화된 Lock



DBCP2 - 각 연결마다 독립적인 Lock → 여러 스레드 동시 접근 가능!

DBCP2




Thread A, B, C 동시 접근 가능!

각 연결마다 독립적인 Lock → Thread A가 conn1 꺼낼 때 Thread B는 conn2 동시에 꺼낼 수 있음

불필요한 직렬화 제거 → 멀티스레드 성능 개선

isValid() 메서드 활용 → SELECT 1 대신 드라이버 레벨 연결 검증

# DBCP2 개선점 정리



## Java 7 기반 완전 재작성

**세분화된 Lock - 전체 Pool 잠금 대신 연결별 독립 Lock**

› Context Switching 비용 감소, 멀티스레드 성능 개선

**isValid() 메서드로 연결 검증 - SELECT 1 오버헤드 제거**

› JDBC 4.0 드라이버 레벨에서 연결 상태 확인, DB 왕복 없음

**Java 7+ 완전 재작성 - try-with-resources, 개선된 동시성 API**

› 레거시 코드 구조 전면 개편, 메모리 누수 방지

# 요약

**DBCP의 레거시 문제를 해결하기 위해**
**Java 7 기반으로 다시 만든 것이 DBCP2다.**

---

| | | |
|---|---|---|
| 멀티스레드 직렬화 | → | **세분화된 Lock** |
| SELECT 1 오버헤드 | → | **isValid() 메서드** |
| 메모리 누수 / 레거시 | → | **Java 7+ 완전 재작성** |

# 4. HikariCP의 등장 배경

# HikariCP 등장



**2012년 Brett Wooldridge — Zero-overhead Connection Pool**

**Connection Pool은 모든 DB 요청마다 거치는 레이어**

› Pool 자체의 오버헤드가 크면 전체 성능이 떨어진다

**"Zero-overhead" - Pool은 최대한 투명하게 동작해야 한다**

**코드 크기 약 165KB - DBCP 수백KB 대비 극소형**

› 코드가 작다 = 클래스 수가 적다 = JIT 최적화가 쉬워진다 = 실행 속도 향상

**Spring Boot 2.x 이후 기본 Connection Pool로 채택**

# 5. HikariCP의 내부 동작 방식

# DBCP2 vs HikariCP



## DBCP2 - Lock 방식 → Context Switching 비용 큼

### DBCP2 (synchronized Lock)

| | |
|---|---|
| **Thread A** | → Lock 획득 시도 → 성공 → 작업 시작 |
| **Thread B** | → Lock 획득 시도 → 실패 |
| **OS** | → Context Switching → Thread B 잠재움 (수 마이크로초 ~ 밀리초 소요) |
| **Thread A** | → 작업 완료 → Lock 해제 |
| **OS** | → Context Switching → Thread B 깨움 |
| **Thread B** | → Lock 획득 → 작업 시작 |

**Context Switching: 수 마이크로초 ~ 수십 마이크로초 / CPU 코어가 많을수록 더 심각**

# DBCP2 vs HikariCP



**HikariCP - CAS 방식 → Context Switching 없음!**

### HikariCP (CAS - Lock-Free)

**Thread A** → CAS 시도 → 성공 → 즉시 작업 시작

**Thread B** → CAS 시도 → 실패 → 잠들지 않고 재시도

**Thread B** → CAS 재시도 → 성공 → 작업 시작

**CAS (Compare-And-Swap) 란?**

기대값과 현재값이 같으면 새 값으로 원자적 교체. 스레드를 잠재우지 않으므로 Context Switching 비용 없음!

# HikariCP 3단계 전략



## Busy Waiting 문제를 단계적으로 해결

1단계

### ThreadLocal - 스레드 전용 저장공간

스레드가 마지막으로 사용한 연결을 **ThreadLocal**에 캐싱 → 다른 스레드와 공유 없음 → 경합 자체가 없음 → **Busy Waiting** 발생 안 함

# HikariCP 3단계 전략



## Busy Waiting 문제를 단계적으로 해결

1단계

### ThreadLocal - 스레드 전용 저장공간

스레드가 마지막으로 사용한 연결을 ThreadLocal에 캐싱 → 다른 스레드와 공유 없음 → 경합 자체가 없음 → Busy Waiting 발생 안 함

2단계

### CopyOnWriteArrayList + CAS

공유 Pool에서 연결 목록 읽기(Lock 없음) → CAS로 연결 상태를 원자적으로 변경 → 성공한 스레드만 그 연결 사용 / 실패하면 다음 연결 즉시 재시도

# HikariCP 3단계 전략



## Busy Waiting 문제를 단계적으로 해결

**1단계**

### ThreadLocal - 스레드 전용 저장공간

스레드가 마지막으로 사용한 연결을 ThreadLocal에 캐싱 → 다른 스레드와 공유 없음 → 경합 자체가 없음 → Busy Waiting 발생 안 함

**2단계**

### CopyOnWriteArrayList + CAS

공유 Pool에서 연결 목록 읽기(Lock 없음) → CAS로 연결 상태를 원자적으로 변경 → 성공한 스레드만 그 연결 사용 / 실패하면 다음 연결 즉시 재시도

**3단계**

### SynchronousQueue + parkNanos

Pool이 완전히 비어있을 때만 사용 → parkNanos()로 나노초 단위 대기 → 반납 즉시 SynchronousQueue로 직접 전달 → Busy Waiting 없이 대기!

# DBCP1 vs DBCP2 vs HikariCP 비교



| 항목 | DBCP1 | DBCP2 | HikariCP |
|---|---|---|---|
| 출시 | 2002년 | ~2013년 | 2012년 |
| Java 기반 | Java 6 | Java 7+ | Java 8 |
| 연결 관리 | synchronized | synchronized | CAS (Lock-free) |
| ThreadLocal | 없음 | 없음 | 있음 |
| 연결 검증 | SELECT 1 | isValid() | isValid() |
| Spring Boot | X | 2.x 이전 | 2.x 이후 |
| 성능 | 낮음 | 중간 | 높음 |

# 6. HikariCP의 적정 Pool 사이즈 공식

# Pool Size 설정의 트레이드오프



### Pool 사이즈가 너무 작으면

요청이 몰릴 때 대기 → **Connection Timeout** 발생 → 사용자 경험 저하, 서비스 장애 위험

### Pool 사이즈가 너무 크면

**DB** 서버 부하 증가 → **Context Switching** 증가 → 오히려 성능 저하 (무조건 크다고 좋은 게 아님)

# 적정 Pool Size 공식 (HikariCP 공식 문서 권장)



### 적정 Pool 사이즈 공식 (HikariCP 권장)

connections = (core_count x 2) + effective_spindle_count

HikariCP 공식 문서 - https://github.com/brettwooldridge/HikariCP/wiki/About-Pool-Sizing

core_count → 서버의 CPU 코어 수
effective_spindle_count → 디스크 스핀들 수 (SSD면 1로 취급)

ex) 4코어 SSD 서버 → (4 × 2) + 1 = 9

# 감사합니다

